Faculty of Science

Department of

Computer Science Final Year Project

Veterinary Clinic Management System

Smart Veterinary Clinic Management System

by

Fares Samir moghazi

Alaa Mohamed Ashour

Omar Mohamed Fares

Mohamed Abdel Aal Sediq Abdel Aal

Acknowledgement

We would like to express our deepest gratitude to our supervisor Dr Mourad Raafat for his invaluable guidance, support, and encouragement throughout the development of our graduation project. His expertise, insights, and constructive feedback were instrumental in shaping the direction of our project and helping us overcome various challenges along the way. We are truly grateful for their mentorship and dedication to our success.

We would like to thank our families for their unwavering support and encouragement. Their patience and understanding during this challenging time were instrumental in allowing us to focus on our work and achieve our goals.

Finally, we would like to express our gratitude to the faculty and staff of our institution for providing us with the resources and opportunities necessary to pursue our academic and professional aspirations. We are honored to have had the opportunity to work on this project and look forward to applying the skills and knowledge we have gained in our future endeavors.

Abstract

Managing veterinary clinics efficiently can be challenging due to the increasing number of patients, visits, and medical records that need to be handled daily. Traditional systems often rely on manual processes or basic digital tools, which can lead to inefficiencies in tracking patient history, managing appointments, and analyzing clinic performance.

This project presents a comprehensive web-based veterinary clinic management system designed to streamline clinic operations and improve decision-making. The system integrates core functionalities such as animal records management, doctor management, visit tracking, and secure user authentication using modern technologies.

Additionally, the system provides an interactive dashboard that offers insightful analytics, including total visits, revenue, and doctor performance. These analytics help clinic staff monitor daily operations and make data-driven decisions efficiently.

Moreover, a data analysis module is implemented to process historical visit data and generate meaningful insights. A time-series forecasting model (ARIMA) is used to predict future clinic cases, enabling better planning and resource allocation. The model is evaluated using performance metrics such as MAE and RMSE to ensure accuracy and reliability.

The system is built using a modern technology stack, including React for the frontend, Flask for the backend, and MySQL for database management, ensuring scalability and performance.

Overall, this project aims to enhance the efficiency of veterinary clinic management by combining system automation, data analytics, and predictive modeling, resulting in a smarter and more effective healthcare solution for animal care.

Table of Contents

Chapter 1: INTRODUCTION .....

1.1 Project Overview .....

1.2 Problem Statement .....

1.3 Objectives .....

1.4 System Scope .....

1.5 Technologies Used .....

1.6 System Architecture Overview .....

Chapter 2: SYSTEM ANALYSIS .....

2.1 Requirements Analysis .....

2.1.1 Functional Requirements .....

2.1.2 Non-Functional Requirements .....

2.2 System Users (Actors) .....

2.3 Diagrams .....

2.3.1 Use Case Diagram .....

2.3.2 Activity Diagram .....

2.3.3 Sequence Diagram .....

Chapter 3: DATA ANALYSIS & MODELING .....

3.1 Dataset Description .....

3.1.1 Data Collection .....

3.2 Data Preprocessing .....

3.3 Time Series Analysis .....

3.3.1 ADF Test .....

3.3.2 ACF & PACF .....

3.4 Forecasting Model .....

3.4.1 ARIMA Model .....

3.5 Model Evaluation .....

3.5.1 MAE .....

3.5.2 RMSE .....

Chapter 4: SYSTEM DESIGN & IMPLEMENTATION 38

4.1 Frontend Implementation (React) .....

4.2 Backend Implementation (Flask) .....

4.3 Database Design (MySQL) .....

4.4 API Endpoints .....

Chapter 5: SYSTEM FEATURES & USAGE .....

5.1 Authentication (Login / Register) .....

5.2 Dashboard & Analytics .....

5.3 Visit Management .....

5.4 Doctor Performance .....

Chapter 6: RESULTS .....

6.1 System Screenshots .....

6.2 Dashboard Results .....

6.3 Model Results .....

Chapter 7: FUTURE WORK .....

Chapter 8: CONCLUSION .....

Chapter 9: REFERENCES .....

1.1 Project Overview

PETS – Veterinary Clinic Management System with Data Analytics and Forecasting

The increasing demand for efficient veterinary healthcare services has highlighted the need for advanced digital solutions to manage clinic operations. Traditional management methods often rely on manual processes or basic systems, which can lead to inefficiencies in handling patient records, tracking visits, and analyzing clinic performance.

The PITS system addresses these challenges by providing an integrated web-based platform that manages all aspects of a veterinary clinic. The system allows users to handle animal records, doctor information, and visit details in an organized and efficient manner. It also provides secure authentication using modern techniques to ensure data protection.

In addition, the system includes an interactive dashboard that visualizes key metrics such as total visits, revenue, and doctor performance. This enables clinic staff to monitor operations in real time and make informed decisions بسهولة.

Moreover, the project incorporates a data analysis and forecasting module that uses time-series techniques to analyze historical visit data. An ARIMA model is applied to predict future clinic cases, helping in better planning and resource allocation. This enhances the overall efficiency and intelligence of the system.

Overall, the system aims to improve veterinary clinic management by combining automation, data analytics, and predictive modeling into a single, user-friendly platform.

1.2 Objectives

The main objective of the PITS system is to develop an intelligent veterinary clinic management platform that improves operational efficiency and supports data-driven decision-making.

- Providing a complete system for managing animals, doctors, and clinic visits efficiently.
- Implementing a secure authentication system using JWT and encrypted passwords to ensure data protection.
- Developing an interactive dashboard that displays key performance indicators such as visits, revenue, and doctor activity.
- Integrating data analysis techniques to extract meaningful insights from clinic data.
- Applying a time-series forecasting model (ARIMA) to predict future clinic cases and support planning.
- Enhancing overall system usability through a modern web interface built with React and Flask.

1.3 Problem Statement

Veterinary clinics often face challenges in managing daily operations efficiently due to the reliance on manual processes or fragmented systems. These traditional approaches make it difficult to maintain accurate records of animals, track visit histories, and monitor clinic performance.

Additionally, the lack of analytical tools prevents clinics from gaining valuable insights into patient trends, doctor performance, and revenue generation. This limits their ability to make informed decisions and plan effectively for future demand.

Moreover, predicting the number of future visits is a complex task without the use of advanced data analysis techniques. Clinics may struggle with resource allocation, leading to either overcrowding or underutilization of staff and facilities.

Therefore, there is a need for an integrated system that not only manages clinic operations efficiently but also provides analytical and predictive capabilities to enhance decision-making.

1.4 System Scope

The PITS system is designed to provide a comprehensive solution for managing veterinary clinic operations. It covers the core functionalities required for handling clinic data and improving workflow efficiency.

The system includes management of animal records, doctor information, and visit details. It allows users to securely access the system through authentication and perform various operations related to clinic management.

Additionally, the system provides analytical features through an interactive dashboard that displays key metrics such as total visits, revenue, and doctor performance. A forecasting module is also included to predict future clinic cases based on historical data.

However, the system is limited to clinic management and data analysis functionalities and does not include external integrations such as online payments or mobile applications in its current version.

1.5 Technologies Used

The PITS system is developed using a modern and efficient technology stack to ensure performance, scalability, and maintainability.

- **Frontend:** React (Vite) is used to build a dynamic and responsive user interface.
- **Backend:** Flask is used to develop RESTful APIs and handle server-side logic.
- **Database:** MySQL is used for storing and managing structured data such as users, animals, doctors, and visits.
- **Authentication:** JSON Web Tokens (JWT) are used for secure user authentication, and bcrypt is used for password encryption.

- **Data Analysis:** Python libraries such as Pandas and Statsmodels are used for data processing and time-series analysis.
- **Machine Learning Model:** ARIMA model is used for forecasting future clinic cases.

1.6 System Architecture Overview

The system follows a client-server architecture that separates the frontend, backend, and database layers to ensure modularity and scalability.

The frontend is developed using React and communicates with the backend through RESTful APIs. The backend, built with Flask, handles business logic, authentication, and data processing. It interacts with the MySQL database to store and retrieve data.

Additionally, a data analysis module is integrated within the backend to process historical data and generate forecasts using the ARIMA model. The results are then displayed on the frontend dashboard in a user-friendly format.

This architecture ensures clear separation of concerns, making the system easier to maintain, extend, and scale in the future.

Chapter 2: System Analysis

2.1 Requirements Analysis

The system analysis phase focuses on identifying and defining the requirements of the veterinary clinic management system. These requirements describe what the system should do and how it should perform under different conditions. They are divided into functional and non-functional requirements to ensure a complete understanding of the system.

2.1.1 Functional Requirements

Functional requirements describe the core functionalities that the system must provide to users.

- The system shall allow users to register a new account.
- The system shall allow users to log in securely using their credentials.
- The system shall authenticate users using JSON Web Tokens (JWT).
- The system shall not allow access to any feature without successful login.
- The system shall allow users to manage animal records (add, update, view).
- The system shall allow users to manage doctor information.
- The system shall allow users to record and manage clinic visits, including diagnosis and cost.
- The system shall display all visit records with related information such as doctor name and visit date.
- The system shall provide a dashboard showing key metrics such as total visits, revenue, and average daily cases.
- The system shall generate analytics such as daily cases and monthly revenue.
- The system shall evaluate doctor performance based on the number of handled visits.
- The system shall support data analysis and forecasting of future clinic cases using a time-series model (ARIMA**).**

2.1.2 Non-Functional Requirements

Non-functional requirements describe the quality attributes and constraints of the system.

- The system shall ensure data security using encrypted passwords (bcrypt).
- The system shall provide secure authentication using JWT tokens.

- The system shall restrict unauthorized access to all system features.
- The system shall be user-friendly and easy to navigate.
- The system shall provide fast response time for all API requests.
- The system shall be scalable to handle increasing data حجمها مع الوقت.
- The system shall ensure data consistency and integrity.
- The system shall be reliable and available during working hours.

2.2 System Users (Actors)

The system includes different types of users who interact with it to perform various operations.

- **Admin / Staff:**
  The Admin (or staff member) is the primary user of the system. They are responsible for managing all system operations, including animal records, doctor data, and clinic visits. The admin also monitors the system through the dashboard and analyzes performance metrics.
- **Doctor:**
  The Doctor is responsible for viewing patient and visit information. This helps in understanding medical cases and supporting clinical decision-making.

2.3 System Description

This section provides an overview of how the system operates and how users interact with it.

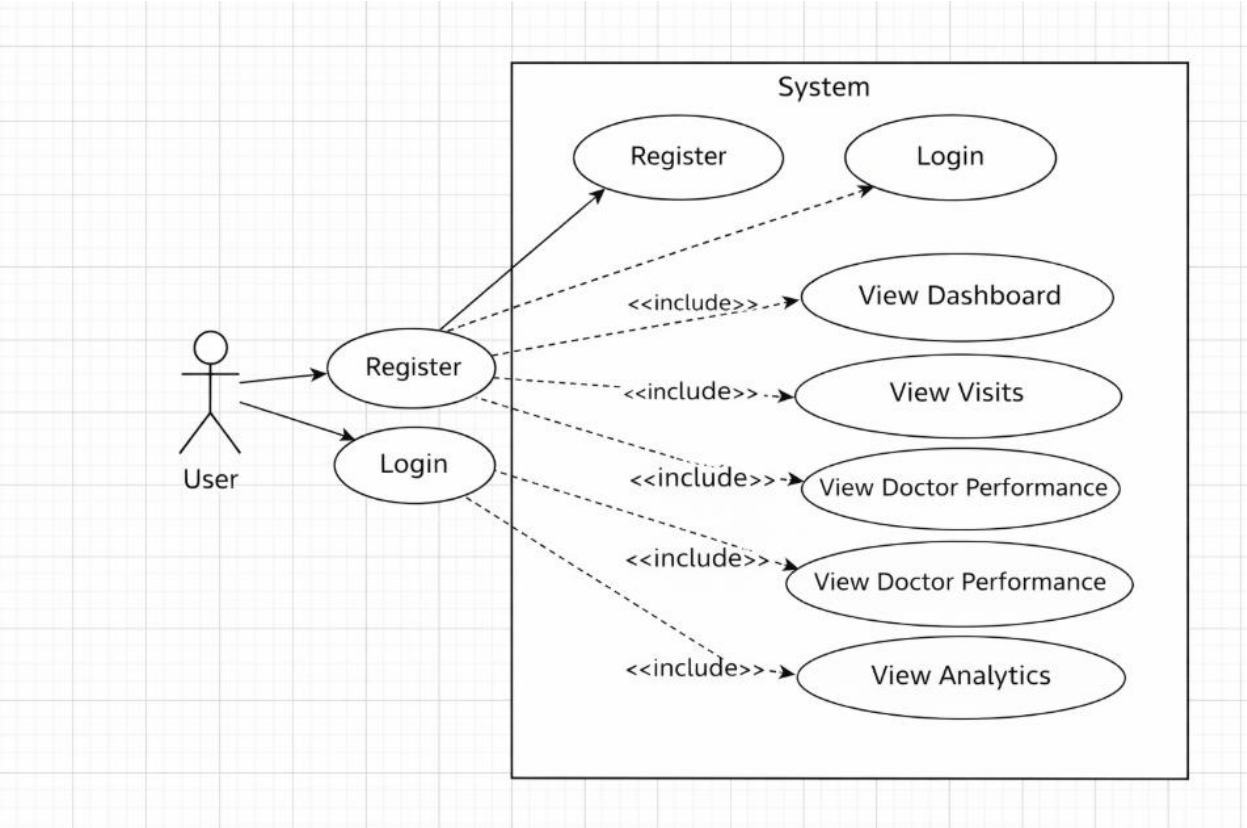
The system is designed as a secure web-based application where authentication is required before accessing any functionality. Users must first log in to the system, and new users must register before gaining access.

Once authenticated, the Admin can manage all aspects of the clinic, including animals, doctors, and visits. The system also provides a dashboard that displays important analytics such as total visits, revenue, and doctor performance.

In addition, the system includes a data analysis module that processes historical visit data and generates forecasts using the ARIMA model. These predictions.

The system ensures that all operations are performed in a structured and secure manner, providing an efficient solution for veterinary clinic management.

Use case diagram:



Chapter 3: Data Analysis & Modeling

3.1 Dataset Description

The dataset used in this project represents veterinary clinic visit records collected from multiple sources. The data includes detailed information about animal visits, doctors, diagnoses, and visit dates.

The dataset was obtained from publicly available sources such as Kaggle and further integrated into the system database. It reflects real-world clinic operations and is used to analyze trends and predict future cases.

Key attributes in the dataset include:

- Visit date
- Animal ID
- Doctor ID
- Diagnosis

- Total cost

3.1.1 Data Collection

The data was collected from external datasets and processed before being inserted into the system database (MySQL). The collected data was initially stored in CSV format and later imported into the database using Python scripts.

The data collection process involved:
- Downloading datasets from Kaggle
- Cleaning and formatting the data
- Converting date fields into a standard datetime format
- Inserting the cleaned data into the Visit table

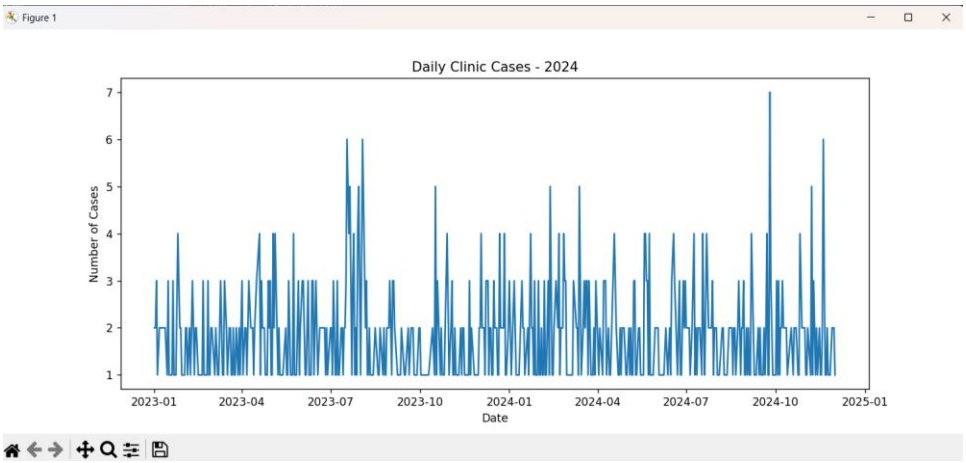
3.2 Data Preprocessing

Before applying any analytical model, the dataset was preprocessed to ensure data quality and consistency.

The preprocessing steps included:
- Converting visit_date to datetime format
- Grouping data by date to calculate daily cases
- Handling missing values by removing incomplete records
- Sorting data chronologically

After preprocessing, the dataset was transformed into a time series format where each date corresponds to the number of daily clinic cases.

Figure 3.1: Daily Clinic Cases



3.3 Time Series Analysis

Time series analysis was applied to study the pattern of clinic visits over time.

3.3.1 ADF Test

ADF results:

- Before: -22.89 (p=0.0)
- After: -13.16 (p=1.27e-24)

3.3.2 ACF & PACF

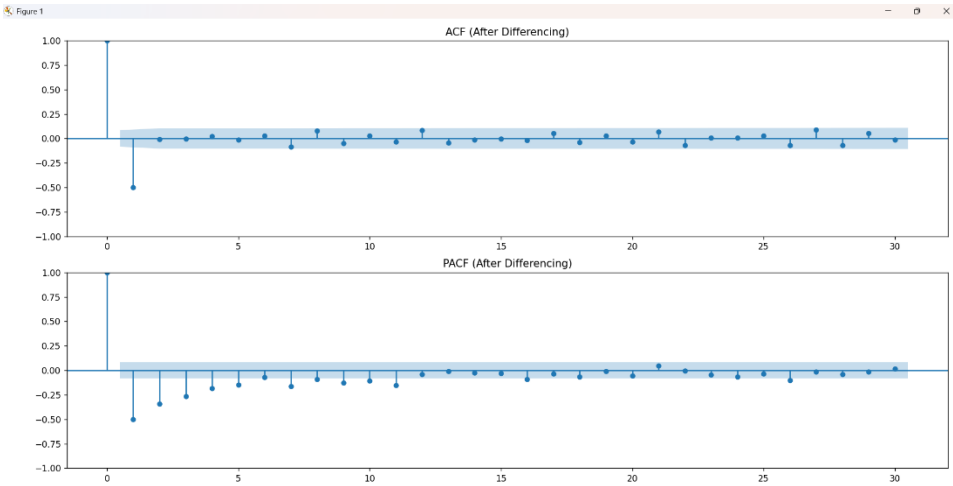
Figure 3.2: ACF & PACF

The ACF (Autocorrelation Function) and PACF (Partial Autocorrelation Function) plots are used to analyze the correlation structure of the time series after differencing.

The ACF plot shows the correlation between the current observation and its previous lagged values. It can be observed that there is a significant negative spike at lag 1, followed by values that gradually fall within the confidence interval. This pattern indicates the presence of a moving average (MA) component in the data.

On the other hand, the PACF plot represents the partial correlation between observations while removing the effect of intermediate lags. The PACF values show a sharp drop after the first few lags and then stabilize within the confidence bounds, suggesting that there is no strong autoregressive (AR) component beyond the initial lags.

Overall, the patterns observed in both ACF and PACF plots suggest that the time series is suitable for an ARIMA model with first-order differencing (d = 1) and a moving average component (q = 1). This analysis supports the selection of the ARIMA(0,1,1) model as an appropriate forecasting model for the dataset



3.4 Forecasting Model

ARIMA models tested:

- ARIMA(1,1,1)
- ARIMA(0,1,1)

Best model: ARIMA(0,1,1)

3.5 Model Evaluation

- MAE: 0.84
- RMSE: 1.22

Figure 3.3: Forecast Results

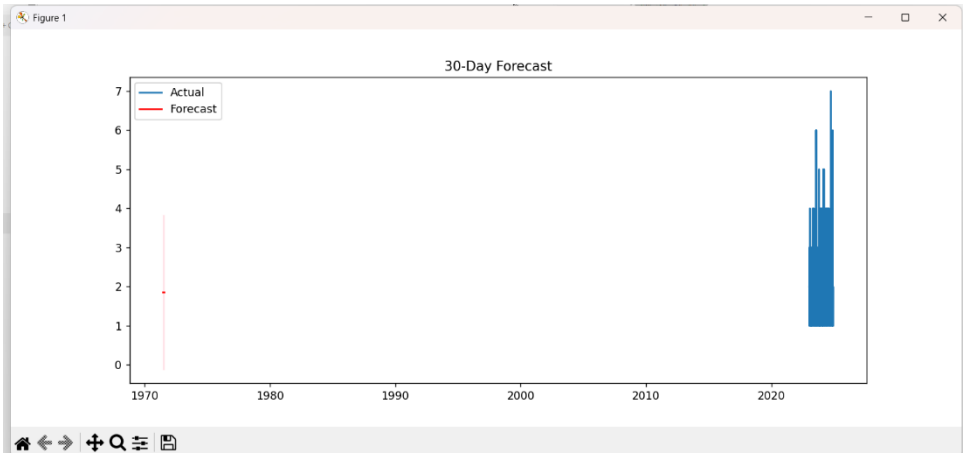
The forecasting results illustrate the predicted number of daily clinic cases for the next 30 days using the ARIMA model. The blue line represents the historical (actual) data, while the red line represents the forecasted values.

From the graph, it can be observed that the model captures the general trend and variability of the data. Although the predicted values are smoother compared to the actual data, they follow the same overall pattern of fluctuations in clinic visits.

The forecast also includes a confidence interval, which represents the uncertainty range of the predictions. This interval indicates that future values are expected to fall within a certain range, rather than being exact values, due to the random nature of real-world data.

Additionally, the model shows stable predictions without extreme spikes, which suggests that the ARIMA model is well-fitted and does not overfit the data. This makes the model reliable for short-term forecasting.

Overall, the forecasting results demonstrate that the model can be effectively used to estimate future clinic demand, helping in resource planning, workload management, and improving operational efficiency within the veterinary clinic



Chapter 4: System Design & Implementation

4.1 Frontend Implementation (React)

The frontend of the system is developed using React (Vite), providing a responsive and dynamic user interface. It communicates with the backend through RESTful APIs to display real-time data.

The system interface is designed to be simple and user-friendly, allowing users to easily navigate between different sections such as the dashboard, visits, and analytics.

Key features of the frontend include:

- A login and registration interface for secure access to the system.
- Dashboard visualization showing key metrics such as total visits, revenue, and average daily cases.
- Data presentation using charts and tables for better understanding of clinic performance.
- Integration with backend APIs to fetch and display real-time data.

The frontend communicates with the backend using HTTP requests and handles authentication using JWT tokens stored securely on the client side.

The frontend handles user authentication and stores the JWT token to allow access to protected routes.

Example of API call from frontend:

```
fetch("http://localhost:5000/dashboard-summary", {
  method: "GET",
  headers: {
    "Authorization": "Bearer " + token
  }
})
.then(res => res.json())
.then(data => console.log(data));
```

This code sends a request to the backend API and includes the JWT token in the headers to authenticate the user.

4.2 Backend Implementation (Flask)

The backend is implemented using Flask and is responsible for handling all system logic, authentication, and database operations.

The backend of the system is developed using Flask, which provides a lightweight and efficient framework for building RESTful APIs.

The backend handles all business logic, data processing, and communication with the database. It is also responsible for authentication, authorization, and data security.

Key backend functionalities include:

- User authentication (login and registration) using JWT.
- Password hashing using bcrypt to ensure security.
- Data retrieval and processing for dashboard analytics.
- Handling API requests for visits, doctors, and system data.
- Generating analytical results such as daily cases and monthly revenue.

The backend is structured in a modular way, making it easy to maintain and extend. It interacts with the database using SQL queries through the PyMySQL library.

Example: User Registration

```
@app.route("/register", methods=["POST"])

def register():

    data = request.json

    username = data.get("username")

    password = data.get("password")

    hashed = bcrypt.hashpw(

        password.encode("utf-8"),
```

```
        bcrypt.gensalt()
    ).decode("utf-8")
    cursor.execute(
        "INSERT INTO user (username,password) VALUES (%s,%s)",
        (username, hashed)
    )
```

This function handles user registration by hashing the password using bcrypt before storing it in the database, ensuring data security.

Example: User Login with JWT

```
@app.route("/login", methods=["POST"])
def login():
    user = cursor.fetchone()
    if not bcrypt.checkpw(
        password.encode("utf-8"),
        user["password"].encode("utf-8")
    ):
        return jsonify({"error": "Invalid credentials"}), 401


    token = create_access_token(identity=str(user["user_id"]))
```

This endpoint verifies user credentials and generates a JWT token, which is used for secure communication between frontend and backend.

4.3 Database Design (MySQL)

The system uses MySQL to store structured data such as users, doctors, and visits.

The system uses MySQL as the primary database for storing and managing structured data.

The database consists of several interconnected tables, including:

- **User Table:** Stores user credentials such as username, password, and role.
- **Animal Table:** Stores information about animals.
- **Doctor Table:** Stores doctor details.
- **Visit Table:** Stores visit records including diagnosis, visit date, and total cost.

The Visit table is the central table in the database, as it links animals and doctors and is used for analytics and forecasting.

Relationships between tables are designed to ensure data consistency and integrity. Foreign keys are used to connect related data across tables.

Example: Visit Query

```
SELECT
    V.visit_id,
    V.animal_id,
    V.doctor_id,
    D.name as doctor_name,
    V.visit_date,
    V.diagnosis,
    V.total_cost
FROM Visit V
LEFT JOIN Doctor D
ON V.doctor_id = D.doctor_id
ORDER BY V.visit_date DESC;
```

This query retrieves all visit records along with doctor names, enabling the system to display detailed visit information.

4.4 API Endpoints

The backend exposes several API endpoints to support system functionality.

The system provides a set of RESTful API endpoints to handle communication between the frontend and backend.

Key API endpoints include:

- **/register (POST):** Used to create a new user account.
- **/login (POST):** Used for user authentication and generating JWT tokens.
- **/dashboard-summary (GET):** Returns key metrics such as total visits and revenue.
- **/daily-cases (GET):** Returns daily case statistics for analysis.
- **/monthly-revenue (GET):** Provides monthly revenue data.
- **/doctor-performance (GET):** Displays the number of cases handled by each doctor.
- **/all-visits (GET):** Retrieves all visit records with detailed information.

All protected endpoints require a valid JWT token to ensure secure access.

Example: Dashboard Summary

```
@app.route("/dashboard-summary")
@jwt_required()
def dashboard_summary():

    cursor.execute("SELECT COUNT(*) as total FROM Visit")
    total_visits = cursor.fetchone()["total"]

    cursor.execute("SELECT SUM(total_cost) as revenue FROM Visit")
    total_revenue = cursor.fetchone()["revenue"]

    return jsonify({
        "total_sales": total_visits,
```

```
        "total_revenue": float(total_revenue)
    })
```

This endpoint calculates and returns key metrics such as total visits and total revenue.

Example: Daily Cases API

```
@app.route("/daily-cases")
@jwt_required()
def daily_cases():

    query = """
    SELECT DATE(visit_date) as visit_date,
           COUNT(*) as cases
    FROM Visit
    GROUP BY DATE(visit_date)
    ORDER BY visit_date
    """

    cursor.execute(query)
    rows = cursor.fetchall()

    return jsonify(rows)
```

This endpoint returns daily case counts, which are used in the frontend for visualization and analysis.

Overall System Implementation

The system integrates frontend, backend, and database components into a unified architecture. The frontend interacts with the backend through secure API calls, while the backend processes data and communicates with the database.

The use of JWT authentication ensures that only authorized users can access system features. Additionally, data analysis and forecasting modules enhance the system's capabilities, making it a powerful tool for veterinary clinic management.

Overall, the implementation demonstrates a complete full-stack system that combines web development, data analysis, and machine learning techniques.

This architecture ensures:

- Separation of concerns between frontend and backend
- Scalability and maintainability
- Secure and efficient data handling

The integration of data analytics and forecasting within the system enhances its functionality, making it not only a management tool but also a decision-support system for veterinary clinics.

Chapter 5: System Features & Usage

5.1 Authentication (Login / Register)

The system provides secure authentication using JWT tokens. Users must log in before accessing any system features. The token is stored in localStorage and used for authorized requests.

The system implements a secure authentication mechanism to ensure that only authorized users can access its features. Authentication is a critical component of the system, as it protects sensitive data related to clinic operations, patient records, and financial information.

The authentication process consists of two main functionalities: user registration and user login.

During the registration process, new users are required to provide a unique username and password. Before storing user credentials in the database, the password is encrypted using the bcrypt hashing algorithm. This ensures that even if the database is compromised, user passwords remain secure and cannot be easily decoded.

Once registered, users can log in to the system by providing their credentials. The system verifies the entered username and password against the stored data. If the credentials are valid, a JSON Web Token (JWT) is generated and returned to the user.

This token plays a crucial role in maintaining secure communication between the frontend and backend. It is stored on the client side (localStorage) and is included in the header of every subsequent API request.

For example, when a user accesses protected endpoints such as the dashboard or visit data, the system checks for the presence of a valid JWT token. If the token is missing or invalid, access is denied, ensuring that unauthorized users cannot interact with the system.

Additionally, the system enforces validation checks on input fields to prevent empty or invalid data submission. Error handling mechanisms are also implemented to provide meaningful feedback in case of incorrect login attempts.

This authentication approach enhances the overall security of the system by combining password encryption, token-based authentication, and access control mechanisms.

Overall, the authentication module ensures that all system operations are performed in a secure and controlled environment, maintaining data integrity and user privacy.

5.2 Dashboard & Analytics

The dashboard displays key metrics such as total visits, revenue, and analytics charts. Data is fetched from backend APIs and visualized using charts.

The dashboard is one of the core components of the system, providing users with a comprehensive overview of clinic performance through interactive visualizations and key performance indicators (KPIs).

The dashboard is designed to present real-time data in a clear and intuitive manner, enabling users to monitor clinic activity and make informed decisions بسرعة وكفاءة.

The system retrieves data from multiple backend API endpoints and displays it using charts and summary cards. These visual elements help users understand trends, patterns, and overall system performance.

Key features of the dashboard include:

- **Total Visits:** Displays the total number of recorded clinic visits within a selected time period.
- **Total Revenue:** Shows the total income generated from clinic visits.
- **Average Daily Cases:** Calculates the average number of daily visits.
- **Peak Day Cases:** Identifies the highest number of visits recorded in a single day.
- **Top Doctor:** Highlights the doctor with the highest number of handled cases.
- **Top Diagnosis:** Displays the most frequent diagnosis among visits.

In addition to KPIs, the dashboard includes several charts for data visualization:

- A **line chart** representing daily clinic cases over time.
- A **bar chart** showing monthly revenue distribution.
- A **doughnut chart** illustrating doctor performance.
- Additional charts such as top diagnoses and recent trends.

The dashboard also includes a time filter that allows users to view data based on different periods such as month, quarter, year, or all-time. This feature provides flexibility in analyzing short-term and long-term trends.

From a technical perspective, the dashboard is implemented using React and Chart.js. Data is fetched using Axios and updated dynamically whenever the selected time period changes.

For example, the system sends API requests such as:

axios.get(`/dashboard-summary?period=${period}`)

axios.get(`/daily-cases?period=${period}`)

These requests retrieve updated data from the backend, which is then processed and displayed in the frontend.

Overall, the dashboard transforms raw data into meaningful insights, making it easier for users to understand clinic performance and support data-driven decision-making.

5.3 Visit Management

The system allows users to view and analyze visits, calculate total revenue, and identify top diagnoses.

The Visit Management module is a central component of the system, responsible for handling and displaying all clinic visit records. It allows users to access detailed information about each visit, including diagnosis, doctor, visit date, and total cost.

This module plays a critical role in organizing clinic data and providing insights into daily operations.

The system retrieves visit data from the backend through a secure API endpoint. Only authenticated users can access this data using a valid JWT token.

For example, the frontend sends a request to fetch all visits:

```
const res = await axios.get("http://localhost:5000/all-visits", {
  headers: { Authorization: `Bearer ${token}` }
});
setVisits(res.data);
```

This request retrieves all visit records from the backend and stores them in the application state for display.

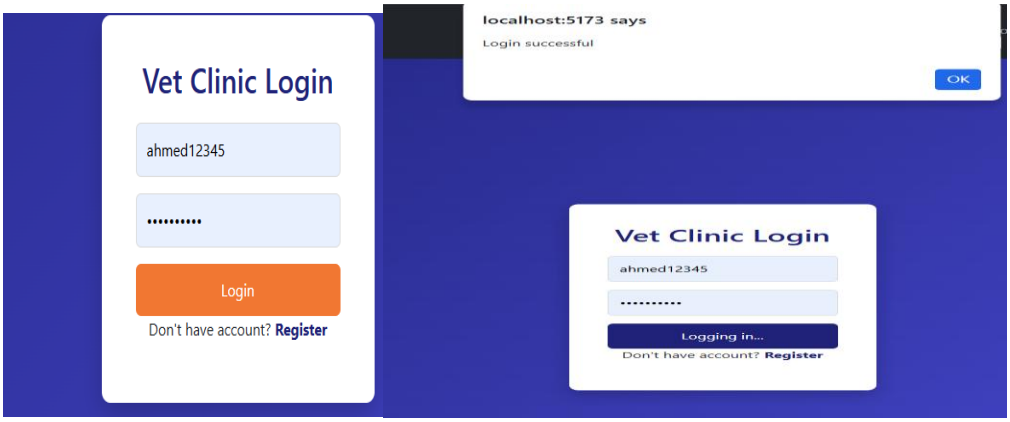
Chapter 6: Results

6.1 System Screenshots

This section presents the visual output of the system through screenshots of the main interfaces. These screenshots demonstrate how the system operates in real-world scenarios and how users interact with different features.

The system interface is designed to be simple, clear, and user-friendly. It includes several main pages such as the login page, dashboard, and visit management page.

- The **Login Page** allows users to securely access the system باستخدام username and password.



- The **Dashboard Page** displays key metrics and charts that summarize clinic performance.

Chapter 7: Future Work

Although the system achieves its main objectives, there are several areas that can be improved or extended in the future to enhance functionality and performance.

One possible improvement is the integration of more advanced machine learning models. While the current system uses the ARIMA model for time series forecasting, future work could include deep learning models such as LSTM (Long Short-Term Memory), which may provide more accurate predictions for complex patterns.

Another enhancement is improving the AI chatbot functionality. The chatbot can be extended to provide more intelligent medical suggestions, support multiple languages, and integrate with external veterinary knowledge bases.

The system can also be expanded to include appointment scheduling features, allowing users to book, update, or cancel appointments directly through the platform.

In addition, implementing role-based access control (RBAC) would enhance system security by assigning different permissions to different user roles such as admin, doctor, and staff.

From a technical perspective, the system can be deployed on cloud platforms such as AWS or Azure to improve scalability, performance, and availability.

Finally, a mobile application version of the system could be developed to allow users to access the system .

Overall, these improvements can significantly enhance the system's capabilities and make it more robust and scalable.

Chapter 8: Conclusion

This project presented a complete veterinary clinic management system that integrates web development, data analysis, and machine learning techniques.

The system successfully provides a user-friendly interface for managing clinic operations, including authentication, visit management, and performance analysis. The dashboard offers clear visual insights into clinic performance, helping users understand trends and make informed decisions.

In addition, the integration of a forecasting model allows the system to predict future clinic visits, which supports better planning and resource management.

The backend, developed using Flask, ensures secure data handling and efficient API communication, while the frontend, developed using React, provides an interactive and responsive user experience.

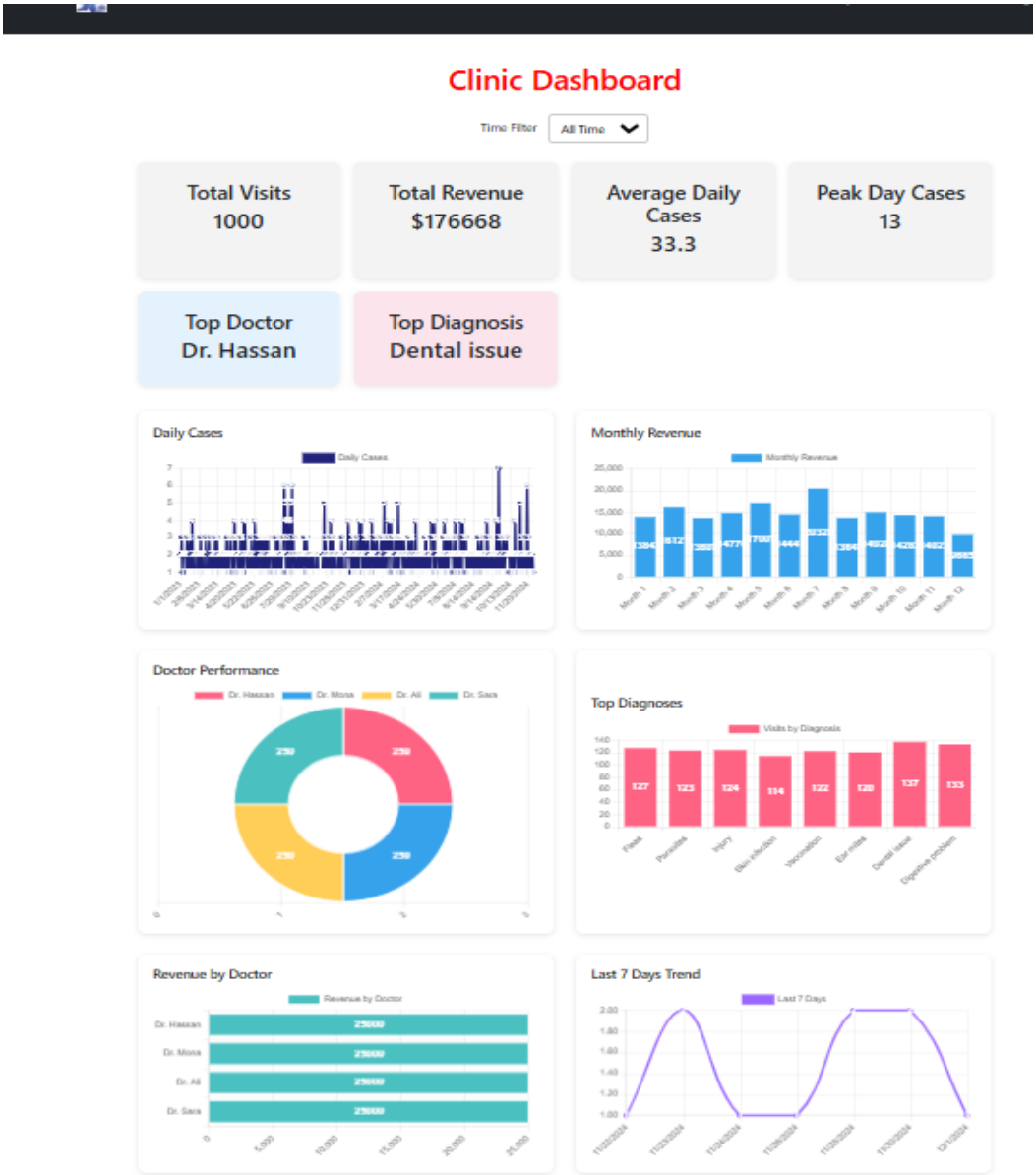
The results demonstrate that the system is effective, reliable, and practical for real-world use in veterinary clinics.

Overall, the project meets its objectives and delivers a comprehensive solution that combines full-stack development with data-driven decision support.

Chapter 9: References

- Kaggle Dataset: Veterinary Clinic Visits Dataset
- Flask Documentation: https://flask.palletsprojects.com/
- React Documentation: https://react.dev/
- Chart.js Documentation: https://www.chartjs.org/
- Statsmodels Documentation (ARIMA): https://www.statsmodels.org/
- JWT Authentication: https://jwt.io/
- Bcrypt Library Documentation: https://pypi.org/project/bcrypt/

These references were used to support the development, implementation, and analysis of the system.



-
- **Dashboard Results Analysis**
- The dashboard provides a comprehensive overview of the veterinary clinic's performance through key performance indicators (KPIs) and visual charts. Each component in the dashboard serves a specific purpose in helping users understand the system data efficiently.
- ---
- **1. Total Visits**
- This metric represents the total number of visits recorded in the system.
- In the displayed result, the total visits reached **1000 visits**.

- This value reflects the overall activity of the clinic.
- A higher number indicates increased demand for clinic services.
- ---
- **2. Total Revenue**
- This metric shows the total income generated from all visits.
- The total revenue in the system is **$176,668**.
- It is calculated by summing all visit costs stored in the database.
- This helps in evaluating the financial performance of the clinic.
- ---
- **3. Average Daily Cases**
- This value represents the average number of visits per day.
- The system shows an average of **33.3 cases per day**.
- It is calculated by dividing total visits by the number of days.
- This metric helps estimate daily workload.
- ---
- **4. Peak Day Cases**
- This metric identifies the highest number of visits recorded in a single day.
- The peak value is **13 cases**.
- It helps detect the busiest day in the clinic.
- Useful for planning staffing and resources.
- ---
- **5. Top Doctor**
- This section identifies the doctor who handled the highest number of cases.
- The top-performing doctor is **Dr. Hassan**.
- This helps evaluate doctor performance and workload distribution.
- ---
- **6. Top Diagnosis**

- This metric shows the most frequent diagnosis.
- The most common diagnosis is **Dental Issue**.
- This insight helps understand common health problems in animals.
- ---
- **7. Daily Cases Chart (Line Chart)**
- This chart shows how the number of daily visits changes over time.
- It displays fluctuations in clinic activity.
- Peaks indicate busy days, while drops indicate low activity.
- Helps identify patterns and trends.
- ---
- **8. Monthly Revenue Chart (Bar Chart)**
- This chart represents revenue distribution across months.
- It shows how income varies monthly.
- Helps identify high-performing months.
- Useful for financial planning.
- ---
- **9. Doctor Performance Chart (Doughnut Chart)**
- This chart compares the number of cases handled by each doctor.
- Each segment represents a doctor.
- Larger segments indicate higher performance.
- Helps in evaluating workload balance.
- ---
- **10. Top Diagnoses Chart (Bar Chart)**
- This chart displays the most common diagnoses.
- It shows frequency of each diagnosis.
- Helps in identifying common diseases.
- Useful for medical analysis.

- ---
- **11. Revenue by Doctor (Horizontal Bar Chart)**
- This chart shows the estimated revenue generated by each doctor.
- It links performance with financial contribution.
- Helps evaluate productivity of each doctor.
- ---
- **12. Last 7 Days Trend (Line Chart)**
- This chart shows recent activity over the last 7 days.
- It highlights short-term trends.
- Useful for quick monitoring of clinic performance.
- ---
- **Overall Analysis**
- The dashboard successfully integrates multiple data sources and presents them in a clear and interactive format. It enables users to:
- Monitor clinic performance in real time
- Analyze trends and patterns
- Make data-driven decisions
- This confirms that the dashboard is an effective tool for both operational monitoring and strategic planning in the veterinary clinic system.
-
- The **Visits Page** provides detailed information about clinic visits, including diagnosis, doctor name, and cost.

These screenshots reflect the successful implementation of the frontend using React and confirm that the system is fully functional.

6.2 Dashboard Results

The dashboard provides a comprehensive overview of clinic performance using real-time data and visual analytics.

The results displayed on the dashboard include:

- **Total Visits:** The total number of clinic visits recorded in the system.
- **Total Revenue:** The total income generated from visits.

- **Average Daily Cases:** The average number of visits per day.
- **Peak Day Cases:** The highest number of visits recorded in a single day.
- **Top Doctor:** The doctor with the highest number of handled cases.
- **Top Diagnosis:** The most frequent diagnosis recorded in the system.

The dashboard also includes multiple charts such as:

- A line chart showing daily cases over time.
- A bar chart representing monthly revenue.
- A doughnut chart illustrating doctor performance.
- Additional charts for diagnosis distribution and recent trends.

These results demonstrate the system's ability to transform raw data into meaningful insights. Users can easily monitor performance and identify trends, which supports better decision-making.

6.3 Model Results

The forecasting model was evaluated using historical clinic visit data. The ARIMA model was applied to predict future daily cases.

The results show that the model successfully captures the general trend of the data. Although some fluctuations exist, the predicted values follow a consistent pattern with the actual data.

The model performance was evaluated using the following metrics:

- **MAE (Mean Absolute Error): 0.84**
- **RMSE (Root Mean Square Error): 1.22**

These values indicate that the model has a relatively low prediction error and provides reliable forecasts.

The forecast graph shows predicted values for the next 30 days along with confidence intervals. This allows users to estimate future clinic demand and plan resources accordingly.

Overall, the results confirm that the model is effective and suitable for practical use in the veterinary clinic system.

Overall Evaluation

The system results demonstrate successful integration between frontend, backend, and data analysis components.

The system provides:

- Accurate data visualization through dashboards
- Efficient management of clinic operations

- Reliable forecasting of future visits

This confirms that the system meets its objectives and delivers a complete solution for veterinary clinic management.